AquaGuard — Bangladesh flood early-warning + IoT pond management

What this is: a final-year capstone project with two connected parts — (1) three machine-learning models that forecast flood risk for 30 river-gauge locations across Bangladesh using only free, public satellite/weather/river data, and (2) an ESP32-based IoT pond-management system (sensors + pumps) that this flood forecasting feeds into. This document is the starting point for anyone new to the project — a professor, a judge, a teammate, or future-you after time away — to get it running and understand what's here.

If you're going to present or demo this project, read this file first, then Current status so you know exactly what's real and what's still in progress — nothing here is oversold.

Project map

Folder / file	What it is
`manuals/`	PDF versions of this file + the setup guides below, for offline/printed reading
`packages/`	The 3 flood model APIs (FastAPI services) — see `packages/SETUP_MANUAL.md` to run all three at once
`frontend-glass/`	The dashboard (glassmorphism look) — **recommended, most up to date version**
`frontend/`	The same dashboard, neumorphic look — kept in sync with `frontend-glass/` feature-for-feature
`hardware/`	The ESP32 + sensors + pumps rebuild — wiring guides, per-sensor test sketches, progress log
`reports/`	One rigorous PDF report per model (methodology, literature comparison, results, limitations)
`backend/`	The training pipeline that produced the models bundled inside `packages/*/models/`
`DECISIONS.md`	Every significant design decision made on this project, why, and what it costs — written for a pre-defense walkthrough
`MODEL_BUILD_PLAN.md`	The full chronological build log — the detailed "how we got here," session by session
`hardware/HARDWARE_LOG.md`	Same idea as above, scoped to the physical hardware rebuild
`hardware/FIREBASE_SETUP.md`	Step-by-step guide for setting up the free cloud database that will link the hardware to the dashboard
`PROJECT_FEATURE_IDEAS.md`	Whole-capstone feature ideas not yet committed to a build plan

---

Running the demo (zero prior context assumed)

Step 1 — Start the 3 model services

```
cd packages
python -m venv .venv
```

Activate it (`.venv\Scripts\activate.bat` on Windows Command Prompt, `.venv\Scripts\Activate.ps1` on PowerShell, `source .venv/bin/activate` on Mac/Linux), then:

```
pip install -r requirements.txt
python run_all.py
```

Leave that terminal open. Full details, troubleshooting, and what each service does: `packages/SETUP_MANUAL.md`.

Step 2 — Open the dashboard

```
cd frontend-glass
python -m http.server 5502
```

Open **http://localhost:5502** in a browser. (Double-clicking `frontend-glass/index.html` directly usually works too — try that first if you'd rather not run a second terminal command; switch to the server method above only if "Use my location" doesn't prompt for permission.)

Step 3 — Walk through it

1. Pick a station from the dropdown (or click "Use my location").
2. Click **"> Run full analysis"** — this calls all 3 model services at once: - **Flood risk classifier** — flood/no-flood risk at 24h/48h/72h ahead - **River discharge forecaster** — predicted river discharge (m^3/s) at the same 3 horizons (its own prediction also feeds directly into the classifier's — see `DECISIONS.md` §8 / `MODEL_BUILD_PLAN.md`'s "3-model pipeline" entry for how) - **Flood susceptibility** — how flood-prone the *terrain* is by nature, independent of current weather
3. Below all three, a **Pipeline summary** appears — 3 plain-language sentences stating what each model actually found (deliberately *not* a single LOW/HIGH verdict — see `MODEL_BUILD_PLAN.md`'s entry on why that design was chosen, so the reader weighs the evidence themselves).

Current status

Written plainly, not to oversell what's finished:

- [done] **All 3 flood models**: trained, tested, documented, each with its own PDF report in `reports/`.
- [done] **Dashboard**: real, live calls to all 3 models; the 3-model pipeline (cascade + combined summary) is built and working.
- [in progress] **Hardware sensor readings on the dashboard**: currently **demo data** (a random walk, clearly labeled as such in the UI) — 4 of 7 physical sensors are individually wired and confirmed working (see `hardware/HARDWARE_LOG.md`), but not yet streaming live to the dashboard.
- [in progress] **Pump control on the dashboard**: currently **simulated** (changes on-screen state only, no real relay control) — the relay hardware itself is sketch-tested but not yet wired to real pumps.
- [in progress] **Live ESP32 → dashboard link + pump automation logic**: in progress. `hardware/FIREBASE_SETUP.md` is the first step (setting up the free database that will carry live sensor data and pump commands between the physical device and the dashboard).
- [in progress] **pH calibration page** (`ph-calibration.html`, linked from the dashboard header): code and firmware side (`hardware/rebuild/07_full_reintegration/AquaGuard_v2.ino`) are written, but need a real Firebase project (see above) and a real ESP32 flash before this actually works end to end.
- [done] **Public hosting (Vercel)**: `frontend-glass/` is live at **frontend-glass-lilac.vercel.app** (static, zero-config, redeploy with `vercel deploy --prod` from that folder). **Important limitation**: the model buttons call

`127.0.0.1:8000/8001/8002` — each *visitor's own machine*, not a shared server — so they only work for whoever has `packages/run_all.py` running locally on the exact device viewing the page (the 3 model services are local-only by design, a deliberate documented choice, see `MODEL_BUILD_PLAN.md`). Anyone else opening the link sees a "couldn't reach service" message on those 3 buttons — that's expected, not broken. For a live demo to someone else, run the services on your own machine first, then share the link (or your screen).

For a technical deep-dive / defense

- **`reports/`** — one PDF per model (`flood-risk-classifier/`, `discharge-forecaster/`, `flood-susceptibility/`), each covering literature review, methodology, results vs. published benchmarks, limitations, and reproducibility steps. `reports/legacy/` holds the original combined pre-split report, kept for history.
- **`DECISIONS.md`** — every consequential decision (data sources, label design, model family, station coverage, etc.), why it was made, what alternatives were considered, and what limitation it leaves — meant to be read start to finish by someone who wasn't in the room.
- **`MODEL_BUILD_PLAN.md`** — the raw, dated, chronological working log of the whole build, for anyone who wants the full "how we actually got here" including dead ends and negative results.